GeoClaw-claude

Python GIS 分析工具集 · 用户手册

版本 v2.2.1 · 2025 年 3 月

UrbanComp Lab · https://urbancomp.net

MIT License · Copyright © 2025 UrbanComp Lab

1. 项目概述

GeoClaw-claude 是由 UrbanComp Lab（https://urbancomp.net）开发的轻量级 Python 地理信息分析工具集，参考 QGIS Processing Framework 的模块化架构设计，专注于城市地理空间数据分析与可视化。

工具集提供从原始 GPS 轨迹数据处理（基于 trackintel）、矢量/栅格空间分析、路网分析，到自然语言驱动的 GIS 操作、跨会话记忆系统的完整工作流，帮助城市计算领域的科研人员与工程开发者高效完成地理信息分析任务。

1.1 核心功能

核心特性	说明
🗣️ 自然语言 GIS	用中文直接驱动空间分析，无需记忆 API 函数名，支持 AI 模式与离线规则模式双引擎
🚶 人类移动性分析	整合 trackintel 框架，提供 GPS 轨迹→停留点→出行段→重要地点完整处理链路与可视化
🧠 跨会话记忆	短期记忆（会话内缓存）+ 长期记忆（持久化知识库），跨任务积累分析知识
🔄 自动更新自检	一键版本检测与升级，全面健康报告（模块完整性、依赖状态）
🗺️ 完整 GIS 工具链	矢量/栅格/路网/制图/坐标转换（WGS84↔GCJ-02↔BD-09）

1.2 版本历史

版本	日期	主要更新
v2.2.1	2025-03	README 重组，移动性分析章节增强，NL 关键词映射修复（出行方式构成图/预测出行方式）
v2.2.0	2025-03	武汉城市 GPS 轨迹测试数据集（37,549 点/5 用户/10 天），完整 Demo 脚本，trackintel 算法来源声明
v2.1.0	2025-03	analysis/mobility 模块（trackintel 集成），NL 新增 10 类移动性操作，测试 20/20
v2.0.0	2025-03	重大升级，自然语言 GIS 平台正式发布，文档与代码全面同步，API 完全兼容 v1.x
v1.3.0	2025-03	自然语言操作系统（nl/ 模块），ask/chat CLI 命令，NL 测试 20/20
v1.2.0	2025-03	自我检测与自动更新（updater.py），check/update/self-check CLI 命令
v1.1.0	2025-03	Memory 记忆系统（短期+长期），memory CLI 命令组（8 个子命令），Memory 测试 37/37
v1.0.0	2025-03	首个正式版本：完整 CLI、Skill 系统、路网/栅格分析、OSM 下载、坐标转换

2. 安装与配置

2.1 系统要求

要求项	说明
Python 版本	Python 3.9 及以上
操作系统	Linux / macOS / Windows（建议 Linux/macOS）
核心依赖	geopandas, shapely, pyproj, rasterio, networkx, osmnx, scipy, matplotlib, folium, trackintel, click
可选依赖	anthropic（Claude AI，用于 NL AI 模式和 Skill 系统）

2.2 快速安装

通过以下步骤安装 GeoClaw-claude：

```
# 克隆仓库
git clone https://github.com/whuyao/GeoClaw_Claude.git
cd GeoClaw_Claude

# 一键安装（推荐）
bash install.sh

# 或手动安装依赖
pip install geopandas osmnx rasterio trackintel anthropic click
pip install -e .
```

2.3 初始化配置

安装完成后，运行初始化向导完成首次配置：

```
geoclaw-claude onboard
```

向导将引导您完成以下配置项：

- 数据目录设置（本地 GIS 数据存储路径）
- Anthropic API Key（用于 NL AI 模式，可跳过，工具在离线规则模式下同样可用）
- 默认坐标系与语言设置

配置文件保存于 ~/.geoclaw_claude/config.json，可随时修改：

```
geoclaw-claude config set anthropic_api_key YOUR_KEY_HERE
geoclaw-claude config show
```

3. 自然语言操作系统

GeoClaw-claude v2.0 的核心特性：用自然语言直接驱动 GIS 分析，无需记忆 API 函数名。系统支持中文及英文输入，并提供 AI 模式（需要 Claude API Key）与规则模式（完全离线）双引擎，自动选择最优解析策略。

3.1 工作原理

处理层	说明
NLProcessor（意图解析）	接收自然语言输入，AI 模式调用 Claude API 理解语义；无 Key 时自动降级为关键词+正则规则模式
ParsedIntent（结构化意图）	包含 action（操作类型）、params（参数）、targets（目标图层）、confidence（置信度）、steps（多步流程）
NLExecutor（GIS 执行）	将意图映射到 geoclaw_claude 分析函数，管理图层上下文，自动写入 Memory 记忆系统
GeoAgent（对话代理）	多轮对话管理，图层上下文跨轮保持，低置信度时主动请求确认

3.2 CLI 用法

单条指令

```
# 空间分析
geoclaw-claude ask "对医院做 1 公里缓冲区"
geoclaw-claude ask "下载武汉市公园数据"
geoclaw-claude ask "加载 hospitals.geojson 然后做 500 米缓冲区然后可视化"
```

```
# 移动性分析
geoclaw-claude ask "读入 data/mobility/wuhan_gps_tracks.csv"
geoclaw-claude ask "一键完成移动性分析"
geoclaw-claude ask "轨迹地图"
```

```
# 调试模式（只解析意图，不执行）
geoclaw-claude ask --dry-run "对医院做核密度分析"
```

```
# 强制指定解析模式
geoclaw-claude ask --rule "裁剪医院到边界范围内"  # 强制规则模式（离线）
geoclaw-claude ask --ai   "叠加医院和地铁站分析服务覆盖"  # 强制 AI 模式
```

交互式多轮对话

```
geoclaw-claude chat          # 自动选择模式
geoclaw-claude chat --ai     # 强制 AI 模式
```

3.3 Python API

```
from geoclaw_claude.nl import GeoAgent

agent = GeoAgent()   # 自动选择 AI / 规则模式
```

```
agent.chat("加载 data/wuhan/hospitals.geojson")
# ✓ 已加载 200 个要素  耗时: 0.12s

agent.chat("对医院做 1 公里缓冲区")
# ✓ 缓冲区完成，200 个要素，半径 1000.0m  耗时: 0.43s

agent.chat("用交互地图显示结果")
# ✓ 地图生成完成

agent.end(title="武汉医院分析")  # 写入长期记忆
```

📌 *当解析置信度低于 60% 时，GeoAgent 会主动向用户确认意图后再执行，避免误操作。*

3.4 支持的操作类型（30+）

操作类别	自然语言示例
数据加载	"加载 hospitals.geojson" / "下载武汉市公园数据"
缓冲区分析	"对医院做 1 公里缓冲区" / "500 米范围内的设施"
叠加/裁剪	"用边界裁剪医院数据" / "叠加分析两个图层"
最近邻分析	"计算医院到地铁站的最近距离"
KDE 核密度	"对医院做核密度分析" / "密度热力图"
分区统计	"按行政区统计医院数量"
路网分析	"计算两点间最短驾车路径" / "做 10 分钟步行等时圈"
坐标转换	"把医院数据从 wgs84 转成 gcj02"
移动性分析	"读入 gps_tracks.csv" / "生成停留点" / "一键完成移动性分析"
移动性可视化	"轨迹地图" / "时间热力图" / "出行方式构成图" / "移动性指标摘要"
可视化制图	"可视化当前结果" / "用交互地图显示"
多步流水线	"读入数据然后生成停留点然后轨迹地图"

4. 人类移动性分析（trackintel）

geoclaw_claude/analysis/mobility/ 模块整合了 trackintel 框架，提供从原始 GPS 轨迹到移动性指标的完整分析流水线，并通过 geoclaw-claude ask 命令支持自然语言调用。

4.1 算法来源声明

项目	信息
GitHub	https://github.com/mie-lab/trackintel
开发团队	Mobility Information Engineering Lab, ETH Zurich (mie-lab.ethz.ch)
版本要求	trackintel ≥ 1.4.2
论文引用	Martin, H. et al. (2023). Trackintel: An open-source Python library for human mobility analysis. Computers, Environment and Urban Systems, 101, 101938.

4.2 数据层级模型

trackintel 定义了从原始 GPS 点到语义地点的五级数据层级：

层级	数据类型	生成算法
Level 0	positionfixes（原始 GPS 点）	read_positionfixes() 读入 CSV / GeoDataFrame
Level 1	staypoints（停留点）	generate_staypoints()：空间阈值（默认 80m）+ 时间阈值（默认 5min）双判定
Level 2	triplegs（出行段）	generate_triplegs()：停留点间的连续移动轨迹，含 predict_transport_mode() 交通方式标注
Level 3	trips（出行）	generate_trips()：活动停留点间的完整出行（A→B）
Level 4	locations（重要地点）	generate_locations()：DBSCAN 聚类（默认半径 120m），可识别家、工作地等语义地点

4.3 自然语言调用

通过 geoclaw-claude ask 命令可以直接使用中文进行移动性分析：

```
# 读入 GPS 轨迹数据
geoclaw-claude ask "读入 data/mobility/wuhan_gps_tracks.csv"

# 逐步分析（精细控制）
geoclaw-claude ask "生成停留点"            # 距离阈值 100m，时间阈值 5min
geoclaw-claude ask "生成出行段"
geoclaw-claude ask "预测出行方式"
geoclaw-claude ask "识别重要地点"
geoclaw-claude ask "识别家和工作地"
```

```
# 一键完成全部层级（推荐）
geoclaw-claude ask "一键完成移动性分析"

# 可视化输出
geoclaw-claude ask "轨迹地图"              # 停留点+出行段+重要地点叠加图
geoclaw-claude ask "时间热力图"            # Weekday×Hour 活动热力图
geoclaw-claude ask "出行方式构成图"
geoclaw-claude ask "移动性指标摘要"        # 回转半径/跳跃距离/覆盖率

# 多步流水线（一句话完成）
geoclaw-claude ask "读入 gps_tracks.csv 然后一键完成移动性分析然后轨迹地图"
```

4.4 Python API

```
from geoclaw_claude.analysis.mobility import (
    read_positionfixes, generate_full_hierarchy,
    identify_home_work, mobility_summary,
    plot_mobility_layers, plot_activity_heatmap,
    plot_modal_split, plot_mobility_metrics,
)

# ① 读入 GPS 数据
pfs = read_positionfixes(
    "data/mobility/wuhan_gps_tracks.csv",
    user_id_col="user_id", tracked_at_col="tracked_at",
    lon_col="longitude", lat_col="latitude",
)

# ② 一键生成完整数据层级
hierarchy = generate_full_hierarchy(
    pfs, dist_threshold=80, time_threshold=5,
    location_epsilon=120, predict_mode=True,
)

# ③ 移动性指标摘要
summary = mobility_summary(hierarchy)
# 包含: n_users, radius_of_gyration_m, jump_length_m, modal_split ...

# ④ 识别家/工作地（OSNA 方法）
locs = identify_home_work(
    hierarchy["staypoints"], hierarchy["locations"], method="osna")

# ⑤ 可视化
plot_mobility_layers(hierarchy, save_path="mobility_map.png")
plot_activity_heatmap(hierarchy["staypoints"], save_path="heatmap.png")
```

4.5 Demo 数据集

data/mobility/ 目录提供武汉城市 GPS 轨迹测试数据，开箱即用：

属性	内容
文件	wuhan_gps_tracks.csv
规模	37,549 个 GPS 轨迹点，5 位用户，10 天（2024-01-15 ~ 2024-01-25）
地域	武汉三镇：汉口 / 武昌 / 汉阳，WGS84 坐标系
用户背景	金融从业者（地铁通勤）/ 互联网工程师（公交）/ 商贸从业者（步行）/ 高校研究员（骑行）/ 医疗从业者（地铁）
处理结果	停留点 ~1,781 | 出行段 ~1,240 | 出行 ~1,240 | 重要地点 ~20

运行完整 Demo（7 步分析，输出 5 张图表）：

```
python examples/wuhan_mobility_demo.py
```

输出图表保存至 output/mobility_demo/，包括：分层移动性地图、出行方式构成图、活动时间热力图（全体用户 + 单用户）、指标综合仪表盘、个人轨迹图。

5. Memory 记忆系统

Memory 系统为 GeoClaw-claude 提供跨任务的知识积累能力，分为短期记忆（ShortTermMemory）和长期记忆（LongTermMemory）两层，通过 MemoryManager 统一管理。

5.1 短期记忆（会话内）

```
from geoclaw_claude.memory import get_memory

mem = get_memory()
mem.start_session("wuhan_hospital_analysis")

mem.log_op("buffer", "hospitals, 1km")    # 记录操作步骤
mem.remember("buf_hospitals", buf_layer)   # 缓存中间结果
layer = mem.recall_short("buf_hospitals")  # 读取缓存
mem.set_context("city", "wuhan")           # 设置上下文变量
```

5.2 长期记忆（跨会话持久化）

长期记忆持久化存储于 ~/.geoclaw_claude/memory/，重启后自动加载：

```
mem.learn(
    title="武汉医院空间分布规律",
    content={"finding": "医院主要集中在三环内，外围覆盖不足"},
    tags=["wuhan", "hospital", "spatial"],
    importance=0.8,   # 重要性 0~1，影响检索排序
)

results   = mem.recall("武汉 医院")         # 关键词检索
recent    = mem.recall_recent(n=5)           # 最近 5 条记忆
important = mem.recall_important(n=5, threshold=0.7)  # 高重要性记忆
```

5.3 会话复盘

```
entry_id = mem.end_session(
    title="武汉医院覆盖分析复盘",
    tags=["wuhan", "hospital"],
    importance=0.8,
)
# 自动记录：操作序列、总耗时、成功/失败统计、中间结果列表
```

5.4 CLI 命令

命令	功能
`geoclaw-claude memory status`	查看记忆系统状态（条目数、磁盘占用）
`geoclaw-claude memory list`	列出所有长期记忆条目

`geoclaw-claude memory list -c knowledge`	按类别筛选（knowledge / analysis / workflow）
`geoclaw-claude memory search "武汉 医院"`	关键词检索记忆
`geoclaw-claude memory learn "标题" "内容"`	手动存入知识
`geoclaw-claude memory forget <entry_id>`	删除指定记忆条目
`geoclaw-claude memory compact`	压缩旧记忆，释放存储空间
`geoclaw-claude memory export -o backup.json`	导出全部记忆为 JSON

6. 自我检测与自动更新

GeoClaw-claude 内置版本管理系统，支持远程版本检测、一键升级和全面健康检测。

6.1 CLI 命令

命令	功能
`geoclaw-claude check`	检测是否有新版本（对比本地与 GitHub 最新版本）
`geoclaw-claude check --json`	JSON 格式输出版本信息（便于脚本集成）
`geoclaw-claude update`	拉取最新代码（git pull）并重新安装（pip install -e .）
`geoclaw-claude update --force`	强制更新（即使版本相同）
`geoclaw-claude update --test`	更新后自动运行全部测试套件进行验证
`geoclaw-claude self-check`	全面健康检测（模块完整性 + 依赖状态 + 版本信息）
`geoclaw-claude self-check --quick`	快速本地检测（跳过网络请求）
`geoclaw-claude self-check --json`	JSON 格式输出健康报告

6.2 Python API

```
from geoclaw_claude.updater import check, update, self_check, print_self_check

# 检测版本
result = check()
# {'current': '2.2.1', 'latest': '2.2.1', 'up_to_date': True}

# 自动更新
update(force=False, run_tests=True)

# 全面健康检测
report = self_check()
print_self_check(report)
# ╔══ GeoClaw-claude Self-Check ═══════════════╗
# ╟ Version      local: v2.2.1  remote: v2.2.1  ✓
# ╟ Modules      18/18 OK
# ╟ Dependencies 12/12 OK
# ╚═══════════════════════════════════════════╝
```

7. 标准 GIS 工具链

7.1 矢量空间分析

```
from geoclaw_claude.analysis.spatial_ops import buffer, clip, nearest_neighbor, kde
from geoclaw_claude.io.vector import load_vector

hospitals = load_vector("hospitals.geojson")

# 缓冲区（自动处理坐标系，支持 meters/km/degrees）
buf = buffer(hospitals, 1000, unit="meters")

# 裁剪
clipped = clip(hospitals, boundary_layer)

# 最近邻距离计算
result = nearest_neighbor(hospitals, metro_stations)
# result 含新列: nearest_id, nearest_dist_m

# KDE 核密度估计
density = kde(hospitals, bandwidth=0.05, grid_size=100)
```

7.2 路网分析

```
from geoclaw_claude.analysis.network import build_network, isochrone, shortest_path

# 下载并构建路网（osmnx 驱动）
G = build_network(bbox=(113.8, 30.5, 114.5, 30.7), network_type="drive")

# 等时圈（以某坐标为中心）
iso = isochrone(G, center=(114.30, 30.60), minutes=[5, 10, 15])

# 最短路径
path = shortest_path(G,
    origin=(114.30, 30.60), destination=(114.40, 30.70),
    weight="travel_time")
```

7.3 栅格分析

```
from geoclaw_claude.analysis.raster_ops import load_raster, slope, reclassify,
zonal_stats

dem = load_raster("dem.tif")
slp = slope(dem)

# 重分类（坡度→坡度等级）
reclass = reclassify(slp, [(0, 5, 1), (5, 15, 2), (15, 90, 3)])
```

```
# 分区统计
stats = zonal_stats(slp, zones_layer, stats=["mean", "max", "std"])
```

7.4 坐标转换

```
from geoclaw_claude.utils.coord_transform import (
    wgs84_to_gcj02, gcj02_to_bd09, transform_layer)

# 单点转换
gcj_lon, gcj_lat = wgs84_to_gcj02(wgs84_lon=114.30, wgs84_lat=30.60)

# 批量图层转换（自动处理所有要素）
gcj_layer = transform_layer(hospitals, src_crs="wgs84", dst_crs="gcj02")
```

📌 *支持全部六个方向互转：WGS84 ↔ GCJ-02 ↔ BD-09，适配国内高德/百度地图场景。*

8. Skill 脚本系统

Skill 系统允许用户编写符合规范的 Python 脚本，封装复杂分析流程，并支持调用 Claude AI API 进行数据解读。Skill 安装后可通过 CLI 一键运行。

8.1 编写 Skill

```
# my_skill.py
SKILL_META = {
    "name":        "hospital_analysis",
    "version":     "1.0.0",
    "author":      "YourName",
    "description": "医院空间覆盖分析",
    "params": {
        "radius_km": {"type": "float", "default": 5.0, "desc": "服务半径（km）"},
        "use_ai":    {"type": "bool",  "default": True,  "desc": "启用 AI 解读"},
    }
}

def run(ctx):
    layer     = ctx.get_layer("input")          # 获取输入图层
    radius_km = float(ctx.param("radius_km"))   # 获取参数
    buf       = ctx.buffer(layer, radius_km * 1000)  # 调用内置分析
    if ctx.param("use_ai"):
        ai_result = ctx.ask_ai("请分析这些数据的空间分布特征，指出覆盖盲区")
        ctx.info(ai_result)
    return ctx.result(output=buf)
```

8.2 安装与运行

```
# 安装到本地
geoclaw-claude skill install my_skill.py

# 查看已安装 Skill
geoclaw-claude skill list

# 运行 Skill（带 AI 分析）
geoclaw-claude skill run hospital_analysis \
    --data hospitals.geojson --ai

# 运行内置 Skill
geoclaw-claude skill run hospital_coverage \
    --data hospitals.geojson
```

9. 完整 CLI 命令参考

9.1 基础命令

命令	功能
`geoclaw-claude onboard`	交互式初始化向导
`geoclaw-claude config set KEY VALUE`	设置配置项
`geoclaw-claude config show`	显示当前配置
`geoclaw-claude test`	运行环境测试（所有模块）

9.2 数据下载

命令	功能
`geoclaw-claude download osm "武汉市" --type hospital`	从 OSM 下载指定类型 POI
`geoclaw-claude download url https://example.com/data.geojson`	从 URL 下载 GeoJSON

9.3 自然语言操作

命令	功能
`geoclaw-claude ask "指令"`	执行单条自然语言 GIS 指令
`geoclaw-claude ask --dry-run "指令"`	解析意图但不执行
`geoclaw-claude ask --rule "指令"`	强制规则模式（离线）
`geoclaw-claude ask --ai "指令"`	强制 AI 模式
`geoclaw-claude chat`	交互式多轮对话（图层上下文跨轮保持）

10. 依赖与项目结构

10.1 核心依赖

包名	版本要求	用途
geopandas	≥ 1.0	核心矢量数据处理
shapely	≥ 2.0	几何运算（缓冲区/叠加/裁剪）
pyproj	≥ 3.5	坐标投影转换

rasterio	≥ 1.3	栅格数据读写与处理
networkx / osmnx	≥ 3.0 / 2.0	路网构建与分析
trackintel	≥ 1.4.2	人类移动性分析（GPS 轨迹处理）
scipy	≥ 1.10	KDE 核密度估计 / 统计分析
matplotlib / folium	≥ 3.7 / 0.14	静态制图 / 交互式地图
click	≥ 8.0	CLI 命令行框架
anthropic（可选）	≥ 0.20	Claude AI 接口（NL AI 模式 + Skill 系统）

10.2 项目目录结构

```
GeoClaw_Claude/
├── geoclaw_claude/
│   ├── __init__.py          # 版本信息（v2.2.1），公共导出
│   ├── cli.py               # CLI 入口（ask/chat/memory/check/update/...）
│   ├── config.py            # 配置系统（JSON + 环境变量覆盖）
│   ├── updater.py           # 版本检测与自动更新
│   ├── skill_manager.py     # Skill 加载与运行
│   ├── core/
│   │   ├── layer.py         # GeoLayer 核心图层类
│   │   └── project.py       # GeoClawProject 项目管理
│   ├── analysis/
│   │   ├── spatial_ops.py   # 缓冲区/裁剪/最近邻/KDE
│   │   ├── network.py       # 路网分析
│   │   ├── raster_ops.py    # 栅格分析
│   │   └── mobility/        # 人类移动性分析（trackintel）
│   │       ├── core.py      # 数据层级生成
│   │       ├── metrics.py   # 移动性指标计算
│   │       └── visualization.py  # 可视化（英文标签）
│   ├── nl/                  # 自然语言操作系统
│   │   ├── processor.py     # NLProcessor（AI + 规则双模式）
│   │   ├── executor.py      # NLExecutor（意图 → GIS 执行）
│   │   └── agent.py         # GeoAgent（多轮对话代理）
│   ├── memory/              # 记忆系统
│   ├── cartography/         # 制图模块
│   ├── io/                  # 数据 I/O
│   └── utils/               # 坐标转换等工具
├── data/
│   ├── wuhan/               # 武汉 GIS 示例数据（7 个 GeoJSON）
│   └── mobility/            # GPS 轨迹测试数据
│       ├── wuhan_gps_tracks.csv   # 37,549 个轨迹点
│       └── README.md
├── examples/
│   └── wuhan_mobility_demo.py     # 完整移动性分析 Demo
├── tests/                   # 测试套件（97/97 通过）
├── docs/                    # 文档（本文件及技术文档）
├── install.sh
└── CHANGELOG.md
```

GeoClaw-claude v2.2.1 · MIT License · © 2025 UrbanComp Lab · https://urbancomp.net